

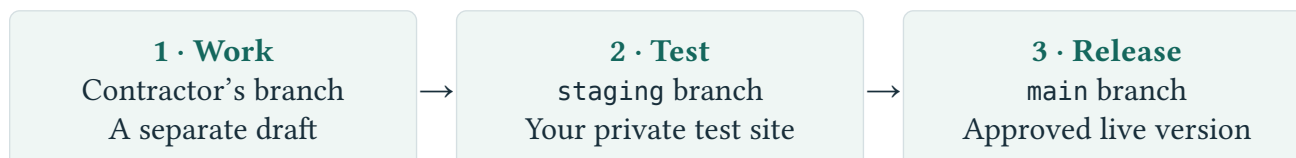
Keep control of your code

A practical guide for a business owner who hires developers

The recommended setup

Contractors propose changes. You try them on a private test site. Only you, or a business representative you choose, can merge them into main. Separate backups let you recover if something goes wrong.

How a change reaches customers



You test the proposed release before merging it into main. The live site uses only an approved version from main; it never follows a contractor's working branch.

Four protections that work together

- **You control the accounts.** GitHub, hosting, the website address, billing, and backups belong to the business.
- **GitHub blocks unauthorized changes to main.** Contractors can work without being repository administrators.
- **The test site is separate.** Testing cannot change customer data or use live passwords.
- **You keep recoverable copies.** Old code and business data are saved somewhere contractors cannot erase.

Five useful words

Repository	The project's code and its saved history.
Branch	A separate line of work, like a draft.
Pull request (PR)	A request to add one branch's changes to another.
Merge	Accept the changes into the destination branch.
Deploy	Put a particular version onto a running site.

Is this enough to prevent permanent loss? It greatly reduces the risk. Branch rules alone are not enough: administrators can change them, approved code can still be harmful, and code history does not back up your customer database. The separate backups on page 6 are essential.

1. Keep the keys to the business

Complete this before giving contractors access. A trusted technical helper can assist while you remain signed in to your own account.

1 Use a business-owned GitHub organization.

An organization is the business’s workspace on GitHub. You own it and control its billing and recovery email. If the repository belongs to a contractor, arrange a transfer into your organization after taking a backup.

2 Choose a plan that enforces these rules.

For the private organization repository described here, use **GitHub Team or Enterprise**. A free organization plan does not provide these private-branch protections. A personal Pro account is a different setup; this guide uses an organization. [2]

3 Give each person their own account and the right access.

In organization settings, set default member access to **None**. In the repository’s **Settings**, open its collaborator/team access page. Add contractors only where needed, with **Write** access. Keep **Admin** and organization **Owner** access with you. Do not share your login. [1]

Who	GitHub access	May merge into main?
Business owner	Organization Owner	Yes, after checks
Contractor	Write on this repository	No, once page 3 is set up
Recovery deputy, optional	Owner, only if fully trusted	Yes; also has full control

An extra administrator also has the power to change protections. If you want **literally only you** to have this power, keep yourself as the only Owner/Admin and store your recovery details safely. [1]

4 Protect sign-in and recovery.

Turn on two-factor authentication: a second sign-in check, preferably a passkey, security key, or authenticator app. Save recovery codes in the business password manager. In organization **Settings** → **Authentication security**, require two-factor authentication for everyone after they enroll. [5]

5 Own the services around the code.

You must control hosting, the domain name, databases, backups, and their bills. Give contractors limited staging access. Review existing people, teams, connected apps, and machine access keys; remove unnecessary access. Replace any owner credentials previously shared with contractors.

Before moving on

Log in yourself and confirm you can manage GitHub, hosting, and backups without asking the contractor.

2. Protect the main branch

Do this with your technical helper. These steps use GitHub’s classic branch protection screen. The helper should check existing rules so a conflicting rule does not undermine this setup. [3]

First, establish the two branches

Take a backup. Put the current approved code on a branch named exactly `main`, preserving its history. In repository **Settings** → **General**, select it as the default branch. Create staging from that same version. Renaming a branch may also require updating the hosting setup. [4]

Then add the main rule

Open repository **Settings** → **Branches**. Add a **classic branch protection rule** with the branch pattern `main`. Configure it as follows, then save. [3]

Protection to configure	Set it to
Changes must go through a pull request	On
At least one approving review is needed	On · 1 approval
New changes invalidate old approvals	On
Passing automated tests are mandatory	On · select the real test checks
The proposed branch must include current main	On
All review conversations must be resolved	On
“Restrict who can push to matching branches”	Only you / your chosen business approvers
“Do not allow bypassing the above settings”	On
People or apps allowed to skip pull requests	Nobody
Permission to dismiss reviews	Only your business approvers
Forced replacement of history; branch deletion	Both off

Why the push restriction matters: merging also updates the branch. Requiring an approval alone does not reserve the merge button for you. Keep contractors and automation apps out of the allowed list, and keep all administrators under business control. [2]

Have the helper first add and run meaningful tests, then select those check names. Select the expected check-producing app where available. Disable automatic merging in repository settings so the owner makes the final decision. Never remove a check just to get a release through.

Protect staging too

Add a separate rule for staging: require pull requests and passing tests; block deletion and forced history changes; apply it to administrators too. Contractors may merge work into staging. Owner approval is required for the later move into `main`.

Check that it actually works

With a contractor account, verify that a harmless test PR cannot be merged into `main`, even after you approve it. Confirm direct edits are blocked. Test destructive attempts only in a disposable repository with the same settings. An owner’s own PR needs another authorized reviewer; authors cannot approve their own work. [2]

3. Keep testing away from live data

Give this page to the person configuring hosting. The hosting service must enforce the separation; branch names by themselves do not do that.

Setting	Staging: test site	Production: live site
Code source	staging	Approved version from main
Who uses it	Owner and contractors; sign-in required	Customers
Database and uploads	Separate, disposable test data	Real data; separate backups
Payments and email	Test payments; email captured or disabled	Real services
Access	Limited contractor access	Business-controlled release access

Ask the helper to configure these five things

1. **Two separate hosting projects or servers.** Set their source branches explicitly. Use separate databases, storage, passwords, and service accounts. Block staging access to live data and backups. A second web address on the same unrestricted server is not sufficient.
2. **A release button you control.** Staging can update automatically. Configure production to wait for your release action in the hosting dashboard. Contractors must not be able to change this policy or deploy through another route. If the host cannot enforce it, use a separate deployment account or system you control.
3. **A visible version number.** Each deployment should show its saved-code ID, called a **commit ID**, in the deployment record or test site. The helper must be able to prove which exact version you tested.
4. **Safe test automation.** Unreviewed code must not receive live passwords, backup access, or an owner's GitHub token. Do not run contractor tests on the production machine. Ordinary repository "secrets" are not hidden from a writer who can change automation. [8]
5. **A working way back.** Save the last working release. Before changes to how data is stored, take a fresh database backup and agree how to recover data as well as code.

If GitHub Actions handles deployment

Under **Settings** → **Environments**, create production. Allow deployment from the **branch** main only, with no tag rule. Keep production credentials inside this restricted environment, not in general repository secrets. The deployment job must use this environment and deploy the approved main version; it must not accept arbitrary contractor code. [6]

Plan limitation: GitHub's built-in environment approval reviewers are unavailable for private repositories on Free, Pro, or Team. On Team, use the owner-controlled hosting approval above. With Enterprise, an environment reviewer can provide that gate. Do not enable a ban on self-review if the sole approver also starts the deployment. [7]

4. Use this routine for every release

Keep one small release in staging at a time. A large batch is harder to test and undo.

1 The contractor prepares the change.

They start a work branch from current main, such as `support/fix-login`, and open a PR into staging. They explain what changed, how to test it, and whether it changes stored data or permissions. Automated tests must pass.

2 The contractor prepares a testable release.

Merge the work into staging and deploy it to the test site. Open a release PR with **destination/base = main** and **source/compare = staging**. It must contain only the agreed work and include the latest main. Record the deployed commit ID in the PR.

3 You test the new work and the old essentials.

Use the checklist below. If anything fails, ask for a fix and repeat the tests. An independent technical reviewer should check sensitive changes, such as sign-in, permissions, payments, data deletion, and deployment scripts. A working screen cannot prove the code is safe.

Your test checklist

- ☐ The requested change works, including an incorrect input.
- ☐ Existing sign-in, search, forms, and other key tasks still work.
- ☐ Ordinary users cannot access someone else's information.
- ☐ Payments, uploads, and email work in test mode, if applicable.
- ☐ The main pages work on a phone and a computer.
- ☐ No unexplained deletions or unrelated changes are in the release.

4 You approve and merge the tested release.

Record "Tested version [commit ID]" in the release PR. In its changed-files review, submit an approval, then use the merge button once checks pass. Do not enable auto-merge. Pause other staging changes during testing. If either branch changes, test the updated result and approve again.

5 You publish that approved version.

A merge may produce a new commit ID. Have automation verify that the final main code matches the tested code. If it differs, deploy it to staging and retest before releasing. Prefer promoting the same tested build package when the host supports it. Release through your hosting account, then check the live site's key tasks.

After release, the helper brings staging up to date with main through a PR, without rewriting history. Keep the release record and previous working deployment. If several jobs must be tested separately, ask for a separate preview site for each PR.

5. Make permanent loss unlikely

A backup only helps if you control it and can restore it. Git history is valuable, but another branch in the same repository is not an independent backup.

Set up backups before the next contractor starts

- **Save code and its history.** Ask the helper to back up every branch and tag, plus any Git LFS large files, into dated archives. A download of the latest files is not a history backup. [9]
- **Save the rest separately.** Back up databases, customer uploads, hosting configuration, and recovery instructions. Include issues, PR records, release files, and a wiki if the business relies on them; a code copy does not include all GitHub information. [9]
- **Keep copies outside the contractor's control.** Use encrypted backups in a separate business-owned storage account; keep recovery keys safe. Contractors and code-running jobs must have no delete access. Use retention locking so old copies cannot be overwritten or removed during their retention period.
- **Keep more than one point in time.** A practical starting policy is daily copies for 90 days, monthly copies for a year, and a copy before each release. Increase frequency if losing a day of data would hurt. Keep another copy offline or with a separate provider. A continuously updated mirror alone can copy deletions too.
- **Prove recovery.** Restore into a separate test location now and every three months. Confirm the application starts, history is present, and sample records and uploads work. Send backup-failure alerts to the owner. Record the recovery time and latest recoverable date.

If something goes wrong

- 1 Stop further damage.**
Pause deployments. Remove suspect access and replace exposed passwords or machine keys. Preserve activity logs and the existing backups. Do not overwrite the only good copy.
- 2 Choose the right recovery.**
For a bad release, restore the previous working deployment and have the helper undo the code change through a PR. For lost or damaged data, restore the database or files separately, accounting for new customer activity since the backup. Reverting code does not restore data.
- 3 Recover a deleted repository.**
The organization owner can check organization **Settings** → **Deleted repositories**. GitHub allows recovery of eligible repositories within 90 days, with exceptions, including some fork situations. Use your independent backup if needed; do not rely on GitHub recovery being available. Recheck permissions and protections afterward. [10]

When a contract ends

Remove GitHub and hosting access, revoke their machine keys and tokens, review connected apps, and replace shared credentials. Confirm the latest backup restores. Removing a GitHub user alone does not revoke every deploy key. Access removal also cannot erase copies of source code they already downloaded. [1]

6. Accept the setup only when it works

Ask the helper to demonstrate this from separate owner and contractor accounts. Test denied destructive actions in a disposable copy, never against your only live repository.

- ☐ You can access GitHub, hosting, billing, and backups independently.
- ☐ The repository is private, on the right plan, with main as default.
- ☐ A contractor can submit work and update staging, but cannot merge into main, edit its protections, or delete the repository.
- ☐ Failed checks and new changes prevent an old approval being used.
- ☐ The staging site cannot use live data, payments, or production credentials.
- ☐ Only an owner-approved main version can reach the live site.
- ☐ You can identify the version tested and the version deployed.
- ☐ A backup restores successfully without contractor access, and the previous working release can be redeployed.

Business owner / GitHub name	
Repository address	
Staging address / live address	
Backup account / recovery helper	
Last successful restore / time taken	

Official references

GitHub documentation checked 21 September 2026. Numbered links throughout this guide are clickable. Plan availability and screen labels may change. Hosting steps depend on your provider; the separation, permissions, and recovery checks still apply.

1. Repository access roles
2. Protected branches and plan availability
3. Set up a branch protection rule
4. Change the default branch
5. Require two-factor authentication
6. Configure deployment environments
7. Deployment approvals and plan limits
8. Protect automation and passwords
9. Back up repository history and large files
10. Restore a deleted repository

The workflow, retention schedule, and acceptance checklist are recommended practices for this scenario. This document explains the setup; it does not configure any repository or server.